VENUSPRO Document Engine — End-to-End 360°

A blueprint for generating and editing policy documents (COI, Declarations, policy jacket, endorsements, cancellation notices, etc.) on top of the existing VENUSPRO metadata-driven platform.

0. Where this plugs into what you already have

Existing piece	Today	Becomes
`FormDefinition` (metadata-driven Form Builder)	Captures input	`DocumentTemplate` — same philosophy, produces output
`usePolicyStore.bindPolicy()`	Sets `documents: ['Policy Declarations']` (just strings)	Emits a `policy.bound` event then generates a real **document package**
`Policy.documents: string[]`	Placeholder labels	`DocumentInstance[]` — real artifacts with status, version, URL
`lib/pdf.ts` (jsPDF, renders the *application*)	Submission PDF	Generalized **render engine** (Dec page, COI, jacket, endorsement)
`PolicyDetail.tsx` documents list (fake "Downloading" toast)	Mock	Live **document inbox** (view / download / e-sign / request COI)
`useStartTransaction` (endorse/cancel/renew/claim)	Opens a form	On submit, generates the matching **transaction document**
Versioning / Audit / Users & Roles pages	Exist	Reused for template versioning, doc audit trail, authoring permissions

The key insight: **a document is just another metadata artifact** — authored once, generated many times by merging policy data into it. That is exactly your form model, inverted.

1. The document universe — what should exist (domain research)

P&C documents organize by **lifecycle stage**. This is the catalog your platform should be able to produce.

Quote / Pre-bind

- **Quote / Proposal letter** — coverage options, limits, indicative premium, subjectivities.
- **Binder** (ACORD 75) — temporary proof of coverage before the policy is issued.
- **Subjectivities / conditions letter** — what is outstanding before bind.

Bind / Issuance (new business) — generated **on bind**

This is the core "policy packet":

1. **Declarations Page ("Dec page")** — the policy-specific cover: named insured, policy number, term, locations, coverage parts, **limits, deductibles, premium**, and the **schedule of forms/endorsements** attached. *(This is the single most important generated doc — it is the only part unique to the policyholder.)*
2. **Policy Jacket / Coverage Form** — the standard contract language: **Insuring Agreement** → **Definitions** → **Exclusions** → **Conditions**. Mostly boilerplate per product, lightly merged.
3. **Schedule of Forms & Endorsements** — the manifest listing every form attached (form number + edition date + title).
4. **Endorsements attached at issuance** — any forms that modify the base coverage from day one.
5. **Certificate of Insurance (ACORD 25)** — one-page proof for third parties (liability). For property: **Evidence of Property** (ACORD 27 personal / 28 commercial). For auto: **Auto ID cards**.
6. **Invoice / premium breakdown / installment schedule** — taxes, fees, surcharges (you already compute surcharges in the rating engine).
7. **Welcome / delivery cover letter**.
8. **State-mandated notices** — fraud warning, OFAC, privacy/data, surplus-lines notice. Jurisdiction-driven.

Mid-term change — generated **on endorsement**

- **Change endorsement document** — describes what changed (coverage added/removed, limit changed, vehicle/location added).
- **Revised Declarations page** — supersedes the prior Dec.
- **Premium-bearing endorsement** — additional or return premium statement (pro-rata).
- **Revised COI** (if certificate holders are affected).

Cancellation — generated **on cancel**

- **Notice of Cancellation (NOC)** — *carrier-initiated*; legally sensitive: typically **30 days advance written notice** (60 in many state amendatory endorsements), reason required, sent to first named insured + scheduled NOC holders.
- **Cancellation confirmation / Policy Release (ACORD 35)** — *insured-requested*.

- **Return / earned premium statement** — short-rate vs pro-rata.
- **Reinstatement notice** — if coverage is restored.

Renewal / Non-renewal

- **Renewal offer + renewal Dec**.
- **Non-renewal notice** — typically **60-120 days advance notice** with reasons.
- **Conditional renewal notice** — renewing but with materially different terms.

Claims (FNOL) — lighter, but in scope

- Claim acknowledgement, reservation-of-rights, status letters.

> **Form-standard anchor:** the ACORD family (25 COI, 27/28 evidence, 35 cancellation/release, 36 agent-of-record, 75 binder, 101 additional remarks schedule) gives you industry-standard layouts to model your built-in templates on.

---

2. The document data model (mirrors `FormDefinition`)

New file `src/data/documents.ts`, designed to read like your existing `types.ts`:

```
export type DocumentType =
  | 'Declarations' | 'PolicyJacket' | 'FormsSchedule'
  | 'Certificate' | 'AutoIDCard' | 'EvidenceOfProperty' | 'Binder'
  | 'Endorsement' | 'CancellationNotice' | 'NonRenewalNotice'
  | 'ReinstatementNotice' | 'Invoice' | 'Quote' | 'RenewalOffer'
  | 'Notice' | 'CoverLetter'

export type DocStatus = 'Draft' | 'Published' | 'Archived'        // template status
export type InstanceStatus =
  | 'Draft' | 'Issued' | 'Delivered' | 'Superseded' | 'Void'      // generated-doc status

// A reusable block — the document analogue of a Field.
export type DocBlock =
  | { id: string; kind: 'heading'; level: 1|2|3; text: string }
  | { id: string; kind: 'richtext'; html: string }               // T&C, clauses, definitions
  | { id: string; kind: 'merge'; token: string; label?: string }  // {{policy.number}}
  | { id: string; kind: 'keyValue'; rows: { label: string; token: string }[] } // dec data
  | { id: string; kind: 'table'; source: 'coverages'|'forms'|'vehicles'|'custom'; columns: GridColumn[] }
  | { id: string; kind: 'clauseRef'; clauseId: string }          // pull from clause library
  | { id: string; kind: 'signature'; party: 'insured'|'carrier'|'agent'; label: string }
  | { id: string; kind: 'image'; src: 'carrierLogo'|'url'; url?: string }
  | { id: string; kind: 'pageBreak' }
  | { id: string; kind: 'conditional'; when: ConditionGroup; blocks: DocBlock[] } // reuse your ConditionGrou

export interface Clause {                // reusable T&C / exclusion / endorsement wording
  id: string; name: string; category: 'Term'|'Exclusion'|'Condition'|'Definition'|'Notice'
  jurisdiction?: string[]                // e.g. ['CA'] — state-specific
  html: string                           // body with {{merge}} tokens
  when?: ConditionGroup                  // auto-include rule
  version: string
}

export interface DocumentTemplate {
  id: string; name: string; type: DocumentType
  carrierId?: string; productId?: string  // same catalog refs as FormDefinition
  version: string; status: DocStatus
  effectiveDate: string; expirationDate: string
  page: { size: 'A4'|'Letter'; margins: number; header?: DocBlock[]; footer?: DocBlock[]; watermark?: string
  blocks: DocBlock[]
}

// A generated artifact attached to a policy (replaces the bare string in Policy.documents)
export interface DocumentInstance {
  id: string; templateId: string; type: DocumentType
  policyId: string; transactionId?: string
  number: string                         // e.g. POL-CA-100482-DEC-001
  status: InstanceStatus; version: number
  generatedAt: string; issuedAt?: string
  supersedesId?: string                  // Dec v1 -> v2 chain
  dataSnapshot: Record<string, unknown>   // frozen merge inputs at issue time
  url?: string                           // blob/object URL or stored path
  deliveries: { channel: 'portal'|'email'|'esign'|'print'; to: string; at: string; status: string }[]
}
```

The merge **context** is built from data you already have: `policy` + `product` + `carrier` + `answers` (from `useFormStore`) + `rating` (from `ratingEngine`) + transaction party. Tokens like `{{policy.number}}`, `{{insured.name}}`, `{{coverages.table}}`, `{{rating.premium}}`, `{{rating.surcharges}}` resolve straight out of your stores.

`Policy.documents` migrates from `string[]` to `DocumentInstance[]` (keep a back-compat getter for labels).

---

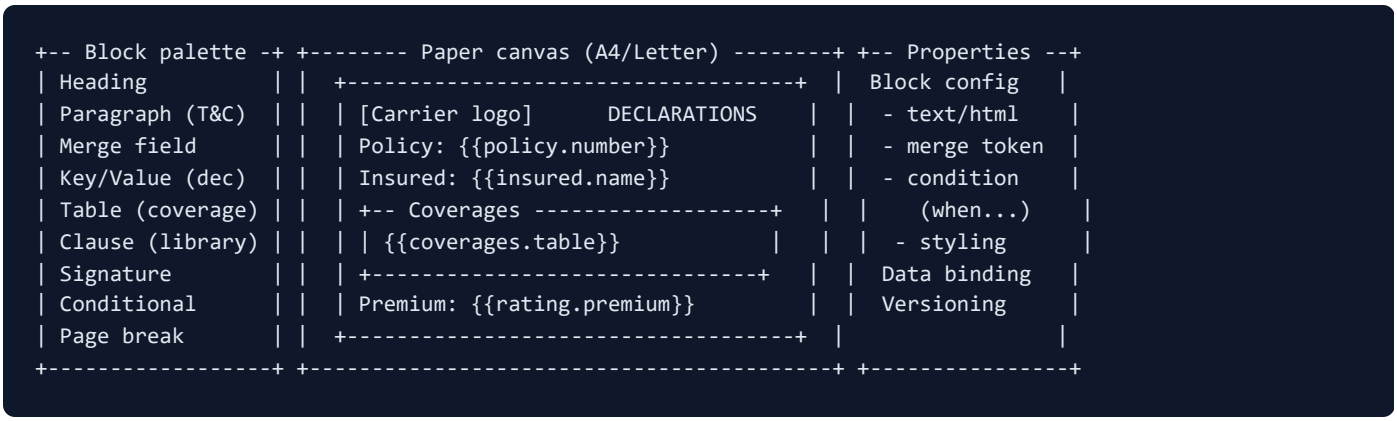
3. The document editing feature (the Template Builder)

This is the "putting terms & conditions, sections, details" piece. Build it as a **sibling of the Form Designer**, reusing your UI primitives (`useReorder`, `Tabs`, `Modal`, `DataTable`, `ConditionEditor`).

New nav group "Documents" with routes:

- `/documents` — **Template Library** (mirrors Form Library: filter by type/product/status/version).
- `/documents/builder/:id` — **Document Builder** (the editor).
- `/documents/clauses` — **Clause Library** (reusable T&C / exclusions).
- `/documents/packages` — **Document Sets** (which docs fire on which event).

The Document Builder canvas uses three panes (same shape as Section/Field Builder):



Editor capabilities:

- **Drag-reorder blocks** (your `useReorder` already does this for fields).
- **Rich-text blocks** for terms/conditions/definitions, with an **inline merge-token picker** (`{{ }}`) that browses the same catalog the forms use.
- **Clause insertion** from the library — author exclusions/conditions once, reuse across templates; clauses carry their own `when` (e.g. auto-include earthquake exclusion only when `state == 'CA'`).
- **Conditional blocks** — reuse your existing `ConditionGroup` + `ConditionEditor` so content shows/hides on answers, coverages, state, or rating output. Same engine that powers form logic.
- **Tables bound to data** — coverage schedule, forms schedule, vehicle/location lists pull live from the policy.

- **Header/footer, carrier logo, page numbering, watermark** (DRAFT/SPECIMEN for previews).
- **Signature placement** (you already have `SignatureField` / `signature` field type — reuse for carrier/insured/agent signatures and e-sign).
- **Versioning + effective dating** (reuse the Versioning page model; issued docs pin to the template version that produced them).
- **Live preview / test-render** against a real bound policy or sample data.

---

4. Configuring endorsement / cancellation / etc. docs (template-to-event wiring)

Extend `Product` (in `products.ts`) with a `documents` map exactly like its existing `forms[]`, and introduce **Document Packages** keyed by event:

```
export interface DocPackage {
  event: 'bound' | 'endorsed' | 'cancelled' | 'nonRenewed' | 'renewed' | 'reinstated' | 'claimOpened'
  templates: { type: DocumentType; templateId?: string; required?: boolean; deliver?: ('portal'|'email'|'esig
}
// On Product:  documents: DocPackage[]
```

Trigger matrix (the "configure endorsement/cancellation docs" answer):

Event (source)	Package generated
`policy.bound` from `bindPolicy()`	Dec + Jacket + Forms Schedule + issuance endorsements + Invoice + COI + state Notices + Cover letter
`policy.endorsed` from endorsement txn	Endorsement doc + **revised Dec** (supersedes prior) + return/additional-premium statement + revised COI
`policy.cancelled` from cancellation txn	NOC *or* Cancellation Confirmation + Return-premium statement
`policy.nonRenewed`	Non-renewal notice (with reason + statutory lead time)
`policy.renewed` from renewal txn	Renewal offer + renewal Dec
`claim.opened` from FNOL	Acknowledgement letter

The `/documents/packages` screen lets an admin assign templates to each event per product — no code change to add a new doc to a product's packet.

---

5. The generation engine

New `src/engines/documentEngine.ts` + `src/lib/documentRender.ts` (generalize today's `pdf.ts`):

```
bindPolicy() / txn submit
        |  emits event + context {policy, product, carrier, answers, rating, party}
        v
documentEngine.generate(event, context)
        |  1. resolve DocPackage for event
        |  2. for each template: load -> buildContext -> resolve merge tokens
        |     -> evaluate conditional blocks / clause `when` (reuse conditionEngine)
        |     -> assemble final block tree
        v
documentRender.toPdf(blocks, page)   // jsPDF now; pluggable HTML->PDF later
        v
DocumentInstance[]  -> freeze dataSnapshot, assign number, status='Issued'
        v
usePolicyStore: attach to policy.documents; supersede prior Dec; audit log entry
```

Engine rules that matter:

- **Snapshot at issue** — an issued doc freezes its merge inputs; later policy edits never silently mutate an issued PDF (legal integrity).
- **Supersession chains** — endorsing a policy creates Dec **v2** with `supersedesId -> v1`; v1 flips to `Superseded`, stays viewable.
- **Immutability** — `Issued` / `Delivered` docs are read-only; corrections require **reissue** (new instance), not edit.
- **Renderer is pluggable** — start with jsPDF (already in repo); Phase 5 swaps in a server-side HTML->PDF path (`server/index.mjs`) for richer T&C typography and true pagination of long jackets.

---

6. Document lifecycle & status model

```
Template:  Draft --publish--> Published --supersede--> Archived
Instance:  Draft --issue--> Issued --deliver--> Delivered
                                |--supersede--> Superseded
                                |--void--> Void   (reissue -> new Draft)
```

- **Who:** Product admin authors templates; UW reviews/overrides the draft packet at bind before issue; system issues; customer receives.
- **Audit:** every state transition writes to your existing **Audit Log** (who/when/what/version).
- **Permissions:** gate authoring/issuing via **Users & Roles** (author template != issue document != void document).

---

7. Delivery & distribution (360 outward)

- **Channels:** portal inbox (default), email, **e-sign** (reuse `SignatureField`), download, print, API webhook.
- **Delivery tracking:** sent / opened / acknowledged — *critical for cancellation & non-renewal* where proof of mailing and lead-time are legally required.
- **COI distribution:** maintain **certificate holders / additional insureds** per policy; reissue/redistribute COIs when coverage changes.
- **API Layer:** expose `POST /policies/:id/documents` (generate) and `GET /documents/:id` (fetch) — fits your existing API Layer page.

---

8. The 360 persona flow (end-to-end)

```
PRODUCT ADMIN - authors DocumentTemplates + Clause library, assigns packages to product/events
        |
UNDERWRITER - at bind, reviews generated draft packet, can override clauses/values -> Issue
        |
SYSTEM (documentEngine) - event-driven generation, numbering, snapshot, audit
        |
CUSTOMER (portal) - receives packet in PolicyDetail inbox; view / download / e-sign;
        |            "Request COI", "Request Change" (-> endorsement -> revised Dec)
        |
THIRD PARTY (cert holder / lender) - receives COI / Evidence of Property; auto-notified on change/cancel
```

Sequence across the policy's life:

`Quote -> Binder -> BIND (full packet issued) -> mid-term Endorsement (revised Dec + endorsement doc) -> COI reissued to holders -> Renewal offer/Dec -> Cancellation (NOC + return premium) -> [optional Reinstatement]`

Each arrow is an event that fires a package, every artifact versioned and audited.

---

9. Compliance & governance (do not skip)

- **Statutory lead times** baked into notice templates/config: cancellation **~30 days** (often 60 by state endorsement), non-renewal **60-120 days** — configurable per state.
- **Mandatory notices** attached by jurisdiction (fraud, privacy, OFAC, surplus lines).
- **e-sign** under ESIGN/UETA — capture intent, audit trail, tamper-evident snapshot.
- **Retention & version pinning** — issued docs reference the exact template + clause versions used.

---

10. Phased build roadmap (mapped to your files)

Phase	Scope	Files
1 — Real docs on bind	`DocumentInstance` model; generate **Dec + COI** on `bindPolicy`; replace string list + fake toast with real download	`data/documents.ts` (new), `store/usePolicyStore.ts`, `lib/documentRender.ts` (from `pdf.ts`), `engines/documentEngine.ts` (new), `pages/customer/PolicyDetail.tsx`
2 — Template Builder	Editor (blocks, T&C, merge picker, conditions), Clause Library, Template Library	new `pages/documents/*`, `nav.ts`, reuse `useReorder` / `ConditionEditor`
3 — Event wiring	`DocPackage` per product/event; endorsement & cancellation generation	`data/products.ts`, `lib/useStartTransaction.ts`, `documentEngine.ts`
4 — Lifecycle & delivery	Supersession, e-sign, delivery tracking, audit, roles, compliance notices	`pages/AuditModule.tsx`, `pages/UsersRoles.tsx`, `SignatureField`
5 — Rich rendering & API	Server-side HTML->PDF, batch, webhooks	`server/index.mjs`, `ApiLayer.tsx`

Recommended starting point

Phase 1 is the highest-impact, smallest slice: turn the placeholder `documents: ['Policy Declarations']` and the fake download toast into a **real Declarations page + COI generated the moment a policy binds**, which makes the whole bind-to-portal chain feel real end-to-end.

Sources

- ACORD Forms (official): https://www.acord.org/forms-pages/acord-forms
- ACORD Forms Index PDF: https://www.acord.org/docs/default-source/forms/forms_index.pdf
- ACORD 25 & 27 explained: https://www.vertikalrms.com/article/acord-25-27-forms-complete-insurance-certificate-guide/
- COI guide to ACORD forms: https://www.getbcs.com/blog/coi-guide-to-acord-forms
- How to read an insurance policy: https://riskcoveragehub.com/how-to-read-insurance-policy-declarations-insuring-agreements-conditions-exclusions/

- Policy Jacket definition: https://www.lopriore.com/insurance-glossary/policy-jacket-meaning/
- Anatomy of an insurance policy: https://www.millersmutualgroup.com/learn/blog/the-anatomy-of-an-insurance-policy/
- Policy cancellation & non-renewal (IIAT): https://www.iiat.org/agency-operations/insurance-laws-regulations/insurance-laws-regulations-most-referenced/policy-cancellation-and-nonrenewal
- Notice of cancellation clauses (IRMI): https://www.irmi.com/term/insurance-definitions/notice-of-cancellation-clauses
- Conditional renewal rules by state: https://uphelp.org/wp-content/uploads/2020/11/boggs-conditional-renewal-rules-by-state.pdf